AI ASSISTED CODING

ASSIGNMENT-8.5

Name: Varsha Sri Chukka

Hall ticket No: 2303A52494

Batch: 50

Lab 8: Test-Driven Development with AI – Generating and

Working with Test Cases

Lab Objectives:

• To introduce students to test-driven development (TDD) using AI code generation tools.

• To enable the generation of test cases before writing code implementations.

• To reinforce the importance of testing, validation, and error handling.

• To encourage writing clean and reliable code based on AI-generated test expectations.

Lab Outcomes (LOs):

After completing this lab, students will be able to:

• Use AI tools to write test cases for Python functions and classes.

• Implement functions based on test cases in a test-first development style.

• Analyze the completeness and coverage of AI-generated tests.

• Compare AI-generated and manually written test cases for quality and logic

Task Description #1 (Username Validator – Apply AI in Authentication Context)

• Task: Use AI to generate at least 3 assert test cases for a

function is_valid_username(username) and then implement

the function using Test-Driven Development principles.

• Requirements:

o Username length must be between 5 and 15 characters.

o Must contain only alphabets and digits.

o Must not start with a digit.

o No spaces allowed.

Example Assert Test Cases:

assert is_valid_username("User123") == True

assert is_valid_username("12User") == False

assert is_valid_username("Us er") == False

Expected Output #1:

• Username validation logic successfully passing all AI-

generated test cases.

Prompt:

#generate a function for username validation is_valid_username, length should be between 5 and 15 and must contain only alphabets and digits and must not start with digit and no spaces allowed. generate few assert test cases

Code:

```python
#task1
#generate a function for username validation is_valid_username, length should be between 5 and 15 and must contain only alphabets and di
def is_valid_username(username):
    if len(username) < 5 or len(username) > 15:
        return False
    if not username.isalnum():
        return False
    if username[0].isdigit():
        return False
    if ' ' in username:
        return False
    return True
# Test cases
assert is_valid_username("user123") == True
assert is_valid_username("123user") == False
assert is_valid_username("user name") == False
assert is_valid_username("us") == False
assert is_valid_username("thisisaverylongusername") == False
print("All test cases passed for is_valid_username.")
```

Output:

```
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding> & C:/Python314/python.
All test cases passed for is_valid_username.
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding>
```

Justification:

This function checks the username length, allowed characters, and starting rule to ensure it follows basic security and readability standards. It prevents spaces and special characters, which are commonly restricted in real systems.

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

• Task: Use AI to generate at least 3 assert test cases for a

function classify_value(x) and implement it using conditional

logic and loops.

• Requirements:

o If input is an integer, classify as "Even" or "Odd".

o If input is 0, return "Zero".

o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

assert classify_value(8) == "Even"

assert classify_value(7) == "Odd"

assert classify_value("abc") == "Invalid Input"

Expected Output #2:

• Function correctly classifying values and passing all test

cases.

Prompt:

#generate a function classify_value that takes a value and classifies it as "Even","Odd","Zero" or "Invalid Input" and generate few assert test cases

Code:

```python
#task2
#generate a function classify_value that takes a value and clas
def classify_value(value):
    if not isinstance(value, int):
        return "Invalid Input"
    if value == 0:
        return "Zero"
    elif value % 2 == 0:
        return "Even"
    else:
        return "Odd"
# Test cases
assert classify_value(0) == "Zero"
assert classify_value(2) == "Even"
assert classify_value(3) == "Odd"
assert classify_value(-4) == "Even"
assert classify_value("string") == "Invalid Input"
print("All test cases passed for classify_value.")
```

Output:

```
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding> & C
All test cases passed for classify_value.
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding>
```

Justification:

This function ensures only integers are processed and avoids unexpected errors. It classifies numbers clearly into zero, even, or odd, which is useful in many logical and mathematical applications.

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

• Task: Use AI to generate at least 3 assert test cases for a

function is_palindrome(text) and implement the function.

• Requirements:

o Ignore case, spaces, and punctuation.

o Handle edge cases such as empty strings and single

characters.

Example Assert Test Cases:

assert is_palindrome("Madam") == True

assert is_palindrome("A man a plan a canal Panama") ==

True

assert is_palindrome("Python") == False

Expected Output #3:

• Function correctly identifying palindromes and passing all

AI-generated tests.

Prompt:

#generate a function is_palindrome that checks if a given string is a palindrome (reads the same backward as forward) and generate few assert test cases

Code:

```
#task3
#generate a function is_palindrome that checks if a given string is a palindrome
def is_palindrome(s):
    s = s.replace(" ", "").lower()  # Remove spaces and convert to lowercase
    return s == s[::-1]  # Check if the string is equal to its reverse
# Test cases
assert is_palindrome("A man a plan a canal Panama") == True
assert is_palindrome("Hello") == False
assert is_palindrome("Madam") == True
assert is_palindrome("No lemon no melon") == True
assert is_palindrome("Python") == False
print("All test cases passed for is_palindrome.")
```

Output:

```
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding> &
All test cases passed for is_palindrome.
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding>
```

Justification:

This function ignores spaces and case differences, making the palindrome check more flexible and realistic. It uses string reversal, which is simple and efficient.

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

• Task: Ask AI to generate at least 3 assert-based test cases for

a BankAccount class and then implement the class.

• Methods:

o deposit(amount)

o withdraw(amount)

o get_balance()

Example Assert Test Cases:

acc = BankAccount(1000)

acc.deposit(500)

assert acc.get_balance() == 1500

acc.withdraw(300)

assert acc.get_balance() == 1200

Expected Output #4:

• Fully functional class that passes all AI-generated assertions.

Prompt:

#generate a function for Bankaccount class with methods deposit, withdraw and get_balance. Ensure that the withdraw method does not allow the balance to go negative and generate few assert test cases

Code:

```python
#task4
#generate a function for Bankaccount class with methods deposit, withdraw an
class BankAccount:
    def __init__(self):
        self.balance = 0

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount

    def withdraw(self, amount):
        if amount > 0 and self.balance - amount >= 0:
            self.balance -= amount

    def get_balance(self):
        return self.balance
# Test cases
account = BankAccount()
account.deposit(100)
assert account.get_balance() == 100
account.withdraw(30)
assert account.get_balance() == 70
account.withdraw(80)  # Should not allow withdrawal
assert account.get_balance() == 70  # Balance should remain unchanged
account.deposit(-50)  # Should not allow negative deposit
assert account.get_balance() == 70  # Balance should remain unchanged
print("All test cases passed for BankAccount.")
```

Output:

```
All test cases passed for is_palindrome.
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding> & C:
All test cases passed for BankAccount.
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding>
```

Justification:

This class models a real bank system by preventing negative deposits and withdrawals beyond the balance. It ensures financial safety and maintains data integrity.

Task Description #5 (Email ID Validation – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for a

function validate_email(email) and implement the function.

• Requirements:

o Must contain @ and .

o Must not start or end with special characters.

o Should handle invalid formats gracefully.

Example Assert Test Cases:

assert validate_email("user@example.com") == True

assert validate_email("userexample.com") == False

assert validate_email("@gmail.com") == False

Expected Output #5:

• Email validation function passing all AI-generated test cases

and handling edge cases correctly.

Prompt:

#generate a function validate_email that checks if a given email address is valid. A valid email should contain exactly one "@" symbol, at least one "." after the "@" symbol, and no spaces. Generate few assert test cases

Code:

```
#task5
#generate a function validate_email that checks if a given
def validate_email(email):
    if email.count("@") != 1:
        return False
    if " " in email:
        return False
    local_part, domain_part = email.split("@")
    if "." not in domain_part:
        return False
    return True
# Test cases
assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("user@.com") == True
assert validate_email("user@com") == False
assert validate_email("user@ example.com") == False
assert validate_email("user@@example.com") == False
print("All test cases passed for validate_email.")
```

Output:

```
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding> & C:/P
All test cases passed for validate_email.
PS C:\Users\matta\OneDrive\Desktop\AI Assisted Coding>
```

Justification:

This function checks the basic structure of an email address, including the presence of one “@”, a domain, and no spaces. It helps detect common formatting mistakes in user input.